Linux网络--HttpCookie与Session

- 1. HTTP 为什么需要 Cookie 和 Session
  - 1.1 会话 Cookie 与持久 Cookie
- 2. Cookie 的读写过程
  - 2.1 Set-Cookie
  - 2.2 Cookie 请求头
  - 2.3 Domain 与 Path
  - 2.4 Expires 与 Max-Age
  - 2.5 Secure、HttpOnly 与 SameSite
- 3. Session 的基本模型
  - 3.1 Session ID 的生成
  - 3.2 Session 存放位置
  - 3.3 登录与注销
  - 3.4 会话轮换
- 4. 会话安全
  - 4.1 会话劫持
  - 4.2 CSRF
  - 4.3 XSS 与 Cookie 窃取
  - 4.4 会话超时与失效
- 5. Cookie、Session 与 Token
- 6. HTTP Session 不是 Linux Session

1. HTTP 为什么需要 Cookie 和 Session

HTTP 请求本身不自动保存上一条请求的业务上下文。用户提交登录表单后，服务器即使验证成功，下一次 GET 请求也不会天然知道“这个请求来自刚才登录的用户”。

Cookie 与 Session 组合解决了这个问题：

```
登录成功
  -> 服务端创建 Session
  -> 返回 Set-Cookie: session_id=随机值
  -> 浏览器保存 Cookie
  -> 后续请求自动发送 Cookie
  -> 服务端用 Session ID 查找用户状态
```

Cookie 是客户端保存和回传小块数据的机制；Session 是服务端保存用户状态的方案。两者常配合使用，但不是同一个东西。

前置阅读：Linux网络--Http表单与动态路由

1.1 会话 Cookie 与持久 Cookie

类型	生命周期
会话 Cookie	未设置过期时间，通常在浏览器会话结束时失效
持久 Cookie	通过 `Expires` 或 `Max-Age` 设置明确存活时间

浏览器恢复上次会话时，可能保留某些会话 Cookie，因此“关闭浏览器就一定删除”不能作为安全保证。

2. Cookie 的读写过程

2.1 Set-Cookie

服务器通过响应 Header 下发 Cookie：

```
HTTP/1.1 200 OK
Set-Cookie: session_id=random-token; Path=/; HttpOnly; Secure; SameSite=Lax
```

浏览器收到后，根据 Domain、Path、Secure、SameSite 和过期时间判断是否保存，以及后续请求是否发送。

一个响应可以有多个 `Set-Cookie` Header，每个 Header 设置一个 Cookie：

```
Set-Cookie: theme=dark; Path=/
Set-Cookie: session_id=random-token; Path=/; HttpOnly; Secure
```

2.2 Cookie 请求头

后续同站请求自动携带：

```
GET /account HTTP/1.1
Host: example.com
Cookie: session_id=random-token; theme=dark
```

Cookie 名称和值之间使用等号，多个 Cookie 之间使用分号和空格分隔。

2.3 Domain 与 Path

`Domain` 决定 Cookie 可以发送给哪些域名，`Path` 限制发送路径。

```
Set-Cookie: token=abc; Domain=example.com; Path=/api
```

不设置 Domain 时通常是 Host-only Cookie，只返回给创建它的主机。现代浏览器会忽略 Domain 前导点，`.example.com` 不应再被理解为与 `example.com` 不同的作用域。

服务器不能通过 Domain 给完全无关的域名设置 Cookie。父域和子域之间的作用范围仍受浏览器和公共后缀规则约束。

2.4 Expires 与 Max-Age

```
Set-Cookie: theme=dark; Max-Age=604800
Set-Cookie: theme=dark; Expires=Wed, 21 Oct 2026 07:28:00 GMT
```

- `Max-Age` 表示相对当前时间的秒数。
- `Expires` 表示绝对时间。
- `Max-Age=0` 或过去的 Expires 可用于删除 Cookie。

同时存在时，现代规范优先使用 `Max-Age`。

2.5 Secure、HttpOnly 与 SameSite

属性	作用	边界
`Secure`	只通过 HTTPS 发送 Cookie	防止明文链路窃取
`HttpOnly`	禁止 JavaScript 直接读取 Cookie	不能阻止 XSS 发起页面内请求
`SameSite=Strict`	跨站请求通常不携带 Cookie	可能影响外部链接跳转体验

属性	作用	边界
`SameSite=Lax`	对跨站请求进行限制	具体行为和浏览器版本有关
`SameSite=None`	允许跨站发送	通常必须同时设置 `Secure`

`HttpOnly` 降低 Cookie 被脚本直接读走的风险，但不能让页面免受 XSS 攻击。`SameSite` 可以缓解 CSRF，但通常还需要 CSRF Token 和来源校验。

Cookie 通常有约 4 KB 的单条大小限制，因此不适合保存大量会话数据。Cookie 中的值也不是自动加密的，不能直接存放密码。

3. Session 的基本模型

服务端为一次登录创建结构化状态：

```
{
  "user_id": 42,
  "username": "alice",
  "created_at": 1760000000,
  "last_seen_at": 1760003600,
  "roles": ["user"]
}
```

客户端只保存对应的 Session ID：

```
session_id = 8f4cf6d...
```

服务器收到 Cookie 后，用 Session ID 查找状态。Session ID 是键，用户状态是值。

3.1 Session ID 的生成

Session ID 必须使用密码学安全随机数，长度和熵应足以抵抗猜测。不能用用户名、时间戳、`rand()` 或简单 MD5 拼接生成安全会话。

不安全的例子：

```
uint32_t randomId = rand() + time(nullptr);
```

```
std::string sessionId = std::to_string(randomId);
```

这类值可预测、范围小，只适合演示机制，不适合真实系统。

安全实现应使用操作系统提供的 CSPRNG，例如 Linux 上的 `getrandom()` 或 `/dev/urandom`，并生成足够长的不可预测令牌。

3.2 Session 存放位置

存储	优点	限制
进程内存	简单、读取快	重启丢失，多实例不能共享
Redis	共享、可设 TTL、性能好	需要额外运维和一致性设计
数据库	持久化、便于审计	每次访问可能有磁盘查询成本
客户端签名 Token	无服务端状态	撤销、权限更新和泄露处理更复杂

Redis 是内存键值数据库，适合保存会话：

```
SET session:8f4cf6d '{"user_id":42}' EX 1800
```

过期时间限制会话长期驻留；服务端仍应支持主动删除。

3.3 登录与注销

登录流程：

1. GET 登录页面。
2. 用户填写账号密码。
3. POST 登录请求。
4. 服务器读取并解析表单正文。
5. 验证密码哈希。
6. 创建随机 Session ID。
7. 保存 Session 状态。
8. 通过 `Set-Cookie` 返回 Session ID。
9. 后续请求携带 Cookie。
10. 服务器查找 Session 并判断登录状态。

注销需要同时做两件事：

```
删除服务端 Session
  + 让浏览器 Cookie 失效
```

只删除客户端 Cookie 不够，因为被复制的 Session ID 仍可在服务端继续使用。

3.4 会话轮换

登录成功后应生成新的 Session ID，不要沿用登录前匿名会话的 ID。这样可以降低会话固定攻击的风险。

以下事件还应考虑轮换或撤销 Session：

- 用户修改密码。
- 用户提升或降低权限。
- 检测到异常设备或异常位置。
- 用户主动退出所有设备。
- 长时间不活动。
- 安全策略规定的绝对过期时间到达。

4. 会话安全

4.1 会话劫持

攻击者获得 Session ID 后，可以尝试冒充用户。防护应组合使用：

- HTTPS 保护传输。
- `Secure` 防止明文连接发送 Cookie。
- `HttpOnly` 降低脚本直接读取风险。
- 登录后轮换 Session ID。
- 支持服务端撤销。
- 监控异常地点、设备和访问模式。

不能只绑定 IP 地址，因为移动网络、代理和正常网络切换都会改变客户端 IP。

4.2 CSRF

CSRF 利用浏览器会自动携带 Cookie 的特点，诱导已登录用户发起非预期请求。

示例风险：

```
<form action="https://bank.example/transfer" method="post">
  <input name="to" value="attacker">
  <input name="amount" value="10000">
</form>
```

防护包括 SameSite Cookie、CSRF Token、Origin/Referer 校验、二次确认和敏感操作重新认证。

POST 本身不能防止 CSRF；浏览器仍可能自动携带登录 Cookie。

4.3 XSS 与 Cookie 窃取

XSS 让攻击者在页面上下文执行脚本。`HttpOnly` 可以阻止脚本直接读取 Cookie，但攻击者仍能在页面内发起请求、读取页面内容或修改 DOM。

因此 Cookie 防护只是整体 XSS 防护的一部分，还需要按输出上下文编码、限制不可信 HTML、配置 CSP 和避免危险 DOM API。

4.4 会话超时与失效

会话至少应有两类时间限制：

- 空闲超时：一段时间无活动后失效。
- 绝对超时：无论是否活跃，超过最长生命周期后失效。

还应在以下情况使 Session 失效：

- 用户注销。
- 用户修改密码。
- 管理员封禁账户。
- 发现高风险行为。
- 用户选择退出全部设备。

5. Cookie、Session 与 Token

方案	状态位置	特点
Cookie 仅保存用户信息	客户端	简单，但泄露后暴露大量隐私，不应保存密码
Cookie + Session ID	ID 在客户端，状态在服务端	撤销能力好，服务端需要存储
签名 Token	Token 自包含状态	易跨服务使用，但撤销和权限更新更复杂

JWT 等 Token 不会自动比 Session 更安全。安全性取决于签名、加密、过期、撤销、存储位置和权限校验。

客户端保存 Token 时也不能把密码放入 Token；若 Token 被窃取，攻击者仍可能冒充用户。

6. HTTP Session 不是 Linux Session

`session` 一词在不同层次中有不同含义：

名称	所属层	含义
HTTP Session	Web 应用	Cookie 或 Token 关联的用户状态
TCP 连接	传输层	两个端点之间的字节流
Linux 登录会话	操作系统	用户登录后关联的终端、进程组和作业

Linux 中的 `jobs`、`fg`、`bg` 管理进程和作业；`Ctrl+C` 向前台进程组发送中断信号，`Ctrl+Z` 暂停前台作业。它们与 Web Session ID 的生成和校验没有关系。

下一篇进入 HTTPS、服务器安全与排障：Linux网络--Http安全部署与排障